

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

Department of Information Technology

Fall 2026-27

Operating Systems Lab BITE303P

PROJECT PROPOSAL

KernelLeakGuard: Detecting and Verifying Linux Kernel Memory Leaks

Submitted by

Om Gunjan Gupta 24BIT0316

Abhishek Kumar Singh 24BIT0309

Aarav Jain 24BIT0023

Under the Guidance of

Dr. ARIVUSELVAN K

Associate Professor, SCORE

VIT, Vellore.

Fall 2026-27

PROJECT PROPOSAL

1. Introduction

Every program needs memory while it works, and it is supposed to give that memory back when it is done. The Linux kernel — the core of the operating system that runs Android phones, most web servers, cloud machines and many embedded devices — is no different. Kernel code asks for memory with functions such as `kmalloc()` and must return it with `kfree()`. If a piece of kernel code forgets to return memory, that memory is gone until the machine is restarted. This is called a **memory leak**.

Leaks in normal applications are annoying; leaks in the kernel are worse. The kernel has no garbage collector and nobody cleans up after it when a program exits, so every leaked byte stays lost. A machine that is meant to run for months slowly loses memory, gets slower and may finally crash. If an attacker can trigger the leaky code again and again, the leak even becomes a way to bring the machine down (a denial-of-service attack).

This project builds **KernelLeakGuard**, a tool that finds such leaks automatically, proves that each one is real by reproducing it inside a running Linux kernel, and suggests a fix. This proposal explains the problem, what we want to achieve, how we plan to do it, which tools we will use and how the work is shared among the three of us.

2. Problem Statement

Kernel functions often stop early when something goes wrong: memory runs out, a device does not answer, a user passes a bad value. These early stops are called **error exits** or **early-exit paths**. On every such exit the function must give back everything it took so far. Programmers usually do this with a ladder of clean-up labels at the end of the function. The example below shows how easy it is to get this wrong: when the third step fails, the code jumps to the wrong label and the memory taken in the second step is never returned.

```
static int sensor_setup(void)
{
        char *rx, *tx, *name;
        int ret;

        rx = kmalloc(RX_SIZE, GFP_KERNEL);       /* take memory #1 */
        if (!rx)
                return -ENOMEM;
        tx = kmalloc(TX_SIZE, GFP_KERNEL);       /* take memory #2 */
        if (!tx) {
                ret = -ENOMEM;
                goto err_rx;
```

```
        }
        name = kasprintf(GFP_KERNEL, "sensor%d", 0);
        if (!name) {                            /* something failed ...      */
                ret = -ENOMEM;
                goto err_rx;                    /* BUG: should be err_tx     */
        }
        ...
err_tx:
        kfree(tx);                              /* give back memory #2 */
err_rx:
        kfree(rx);                              /* give back memory #1 */
        return ret;
}
```

Such bugs are hard to find for three reasons. First, error exits run only when something fails, which almost never happens during normal testing, so the bug can sit in the code for years. Second, the kernel is enormous (tens of millions of lines), so reading it by hand is impossible. Third, existing automatic tools either read the code and produce many false alarms that a person must check one by one, or run the code and only find leaks on the paths they happen to reach.

Problem statement: design and build a tool that automatically finds memory leaks on the error exits of Linux kernel code, confirms each suspected leak with evidence from a real running kernel instead of leaving it to a human, and proposes a checked fix.

3. Motivation

Memory leaks on error exits are one of the most common bug types fixed in the Linux kernel; the research tool MLEE alone reported 120 new ones. Each is small, but together they matter, because the kernel runs on billions of devices that are rarely restarted. We also found the topic a very good fit for an Operating Systems lab: it touches memory management, kernel modules, error handling, virtual machines and the debugging tools built into Linux itself, and it lets us work with real kernel code instead of a simulation.

- **Reliability:** servers and phones run for weeks; a slow leak eventually causes out-of-memory failures.
- **Security:** a leak that a user can trigger repeatedly can exhaust the kernel's memory (denial of service).
- **Developer time:** a tool that only says “maybe a leak here” still leaves hours of manual checking; proof attached to each report saves that time.
- **Learning:** the project makes us work directly with the kernel's memory allocator, module loading and fault injection — core OS topics.

4. Objectives

1. **Find:** build a static analyser that reads compiled kernel module code (LLVM IR) and points out memory that is not given back on some error exit, together with the failing call that leads there.
2. **Prove:** build a runtime checker that boots our own Linux kernel in a virtual machine, makes exactly that call fail using the kernel's fault-injection feature, and asks the kernel's own leak detector (kmemleak) whether memory was really lost.
3. **Decide:** label every warning as confirmed, not reproduced, or inconclusive, and keep the evidence (kernel messages, stack traces) with it.
4. **Fix:** generate a patch for each confirmed leak and check it: it must compile, the warning must disappear and the leak must be gone when tested again.
5. **Measure:** build a test set of kernel modules with known leaks and known safe code, and compare our tool with simpler approaches and an existing free tool.
6. **Show:** provide a command-line tool, a web dashboard and clear documentation so that others can use and check the results.

5. Scope

To keep the project achievable in one semester, we set clear limits.

Table 5.1: Project scope

In scope	Out of scope
Memory leaks on error exits of Linux kernel modules (C code)	Other bug types (use-after-free, races, deadlocks), except when we happen to find them
Kernel memory functions: kmalloc, kzalloc, vmalloc, kmem_cache, kstrdup and similar	Leaks of other resources (file handles, locks, reference counts) in general
Our own test modules and real modules that need no special hardware	Drivers that need real hardware to load
Linux 6.18 running in QEMU on a laptop (WSL2 on Windows)	Running on physical machines or other operating systems
Error exits reached while a module loads (and test triggers in our modules)	Code reached only by mounting disks or special system calls (future work)

6. Background and Literature Survey

6.1 Background

Kernel memory allocation. Kernel code gets memory from the slab allocator (`kmalloc()`, `kzalloc()`, `kmem_cache_alloc()`) or from `vmalloc()` for large areas, and returns it with `kfree()`, `vfree()` or `kmem_cache_free()`. Unlike user programs, there is no automatic clean-up.

kmemleak. Linux has a built-in leak detector. It remembers every allocation and, when asked, scans the kernel's memory for pointers to each one. An allocation that nobody points to any more is reported as an "unreferenced object", together with the call stack that allocated it. This is our source of proof.

Fault injection. Linux can deliberately make allocations fail, for testing. The file `/proc/self/fail-nth` lets a program say "make the n-th allocation of my next system call fail". This lets us steer the kernel onto exactly the error exit we want to test.

LLVM. The kernel can be compiled with clang into LLVM IR, a simple, well-defined form of the program that is much easier to analyse than C source. Our analyser works on it.

6.2 Existing work

Table 6.1: Existing approaches

Approach	Example	Strength	Weakness
Read the code (static analysis)	MLEE (2021)	Finds leaks on paths that never run; found 120 real leaks	Many false alarms; a person must check each warning
Ownership reasoning	K-MELD (2021)	Understands who owns memory	Static only; no proof
Pattern checkers	Coccinelle, Smatch, cppcheck	Fast, easy to run	Only find simple patterns; miss leaks that cross functions or branches
Run the code (fuzzing)	syzkaller + kmemleak	Real evidence	Only finds what it happens to reach; no explanation of the cause
Fail every allocation	fail-nth sweeps	Systematic	Very many runs; slow

6.3 Gap we address

No existing tool combines the two sides: static tools find candidates but cannot prove them, and dynamic tools prove leaks but cannot aim. Our idea is to let the static analyser **aim** and the running kernel **prove**: the analyser says exactly which allocation leaks, on which exit, and

which failing call leads there; the virtual machine then makes only that call fail and kmemleak gives the verdict.

7. Proposed Method

KernelLeakGuard works in three stages, shown in the architecture figure below.

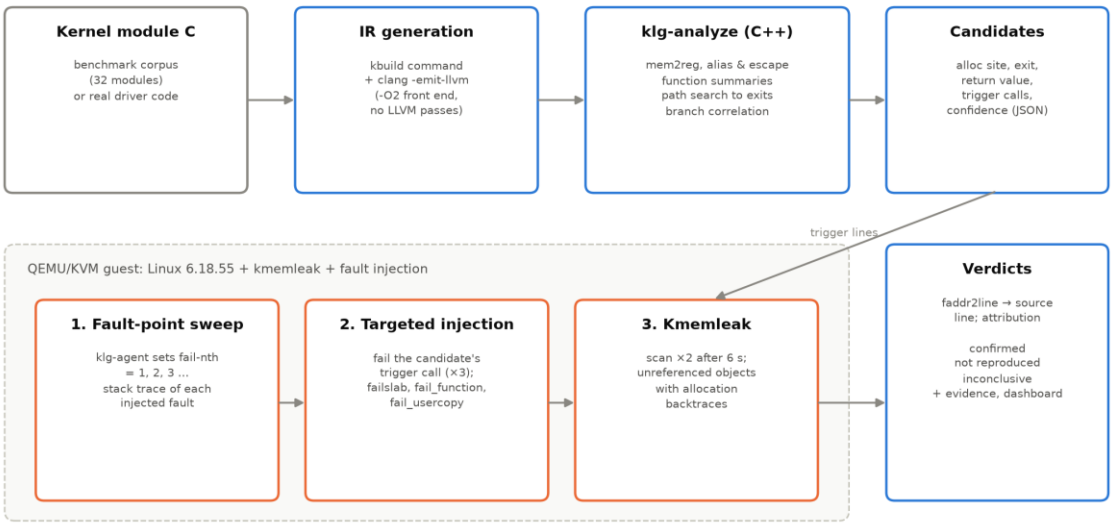


Figure 7.1: Planned architecture of KernelLeakGuard

7.1 Stage 1 — Find (static analysis)

- Compile each kernel module with clang into LLVM IR, using the kernel's own build flags.
- For every memory allocation, follow the pointer through copies, function calls and local structures.
- Search every path from the allocation to each return statement. If a path neither frees the memory nor hands it over to someone else (for example by storing it in a global list), report a possible leak.
- Remember which call failed on that path (the "trigger") and the error that is returned.

7.2 Stage 2 — Prove (runtime verification)

- Build our own Linux kernel with kmemleak and fault injection switched on, and boot it in QEMU (a virtual machine), so failures cannot harm the laptop.
- Inside the virtual machine, a small agent program loads the module and fails the allocations one by one (1st, 2nd, 3rd, ...) to learn which failure number belongs to which line of code.

- Then it fails exactly the trigger found in stage 1, three times, and asks kmemleak for unreferenced objects.
- If an object allocated at the reported line is left behind, the warning is **confirmed**; if the path ran but nothing leaked, it is **not reproduced**; if the failure could not be triggered, it is **inconclusive**.

7.3 Stage 3 — Fix and report

- For a confirmed leak, add the missing free at the right place (usually by re-using an existing clean-up line), compile the module, analyse it again and re-run the test.
- Show everything in a command-line report and a web dashboard with the source code, the failing call and the kernel's evidence.

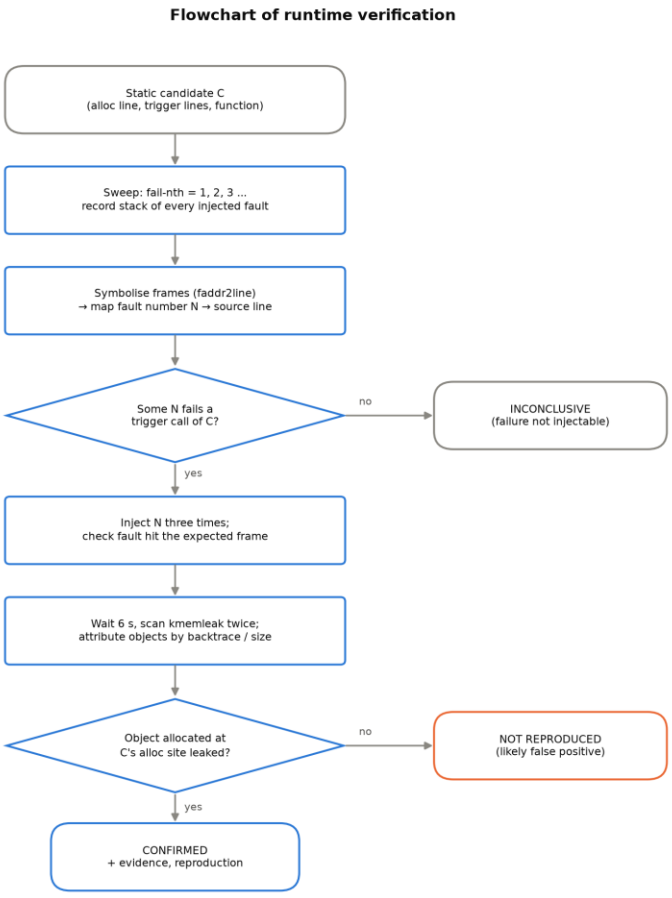


Figure 7.2: Planned flow of the runtime verification

8. Operating System Concepts Involved

Table 8.1: Operating-system concepts

OS concept	How the project uses it
Memory management	Kernel allocators (slab, kmalloc, vmalloc, kmem_cache), freeing memory, and how kmemleak scans memory to find lost objects
Kernel modules	Loading and unloading modules (insmod, finit_module), module initialisation and exit code
Error handling	Error codes such as -ENOMEM, clean-up ladders with goto, error exits
Virtualization	Running a separate Linux kernel in QEMU, sharing files with the host (9p), serial console
System calls and /proc, debugfs	fail-nth in /proc, kmemleak and fault-injection controls in debugfs
Linux system programming	A C agent that loads modules and drives fault injection; shell scripts for the guest
Performance analysis	Counting how many kernel runs each method needs; precision and recall

9. Tools and Software

Table 9.1: Tools and software

Tool	Purpose
Linux 6.18 (LTS) source	The kernel we build, test and analyse
Clang / LLVM 21	Compile the kernel and produce LLVM IR; library for our analyser
C++17	The static analyser (klg-analyze)
C	Test modules and the in-guest agent (klg-agent)
QEMU 10	Virtual machine that runs our kernel
Python 3	Orchestration, verdicts, evaluation, reports
Streamlit, pandas, matplotlib	Web dashboard and charts
BusyBox	Tiny root file system inside the virtual machine
cppcheck	Existing tool we compare against
Git	Version history and backup
Ubuntu on WSL2 (Windows 11)	Development machine

Hardware: one laptop (8 or more CPU cores, 16 GB RAM, about 10 GB free disk). No special devices are needed.

10. Evaluation Plan

To know whether the tool really works, we will write our own set of small kernel modules where we know the right answer: some contain leaks (marked as such in the source) and some contain tricky but correct code that a naive tool would flag. We will then count:

- **True positives:** real leaks the tool reports.
- **False positives:** safe code the tool wrongly reports.

- **False negatives:** real leaks the tool misses.
- **Precision and recall:** the share of reports that are correct, and the share of leaks that are found.
- **Cost:** how many times the kernel code had to be run.

We will compare: our static analysis alone, the complete tool, a naive "missing-kfree" check, failing every allocation blindly, random fault injection, and cppcheck. If time allows we will also run the tool on real Linux modules.

11. Expected Outcomes

- A working tool that finds error-exit leaks and proves or refutes each warning.
- A benchmark of kernel modules with known answers and a comparison with other methods.
- Generated and checked fix patches.
- A dashboard, a user manual, a final report and a presentation.

12. Work Plan

Table 12.1: Work plan

Week	Work	Result
1	Study kmemleak, fault injection, MLEE; set up WSL2, clang, QEMU	Environment ready
2	Build Linux 6.18 with debugging features; boot it in QEMU	Kernel boots in the VM
3	Write the benchmark modules and the guest agent	Modules load in the VM
4	Static analyser: allocations, paths, frees	First warnings
5	Ownership/escape handling, function summaries	Fewer false alarms
6	Fault-point mapping and targeted confirmation	First confirmed leak
7	Verdicts, evidence, symbolising stack traces	Review-I prototype
8	Baselines (exhaustive, random, cppcheck), metrics	Comparison tables
9	Fix patch generation and validation; dashboard	Patches and dashboard
10	Real kernel modules; testing; documentation	Final report, manual, slides

Milestones: **Review-I** — the core pipeline runs end to end on the benchmark (about 60–70% complete). **Review-II** — complete tool, evaluation, dashboard and documentation.

13. Team Members and Responsibilities

Table 13.1: Team responsibilities

Member	Register no.	Responsibilities
Om Gunjan Gupta	24BIT0316	Static analyser in C++/LLVM: allocation tracking, ownership, path search, triggers, fix locations

Abhishek Kumar Singh	24BIT0309	Kernel build, virtual machine, guest init, fault-injection agent, module loading and symbol logging
Aarav Jain	24BIT0023	Benchmark modules and ground truth, Python verifier and verdicts, evaluation and baselines, dashboard

All three of us will test, write documentation and prepare the presentation together, and each will be able to explain every part during the reviews. Guide: **Dr. Arivuselvan K, Associate Professor, SCORE**.

14. Risks and How We Will Handle Them

Table 14.1: Risks

Risk	What we will do
Kernel crashes while we inject failures	Run everything in a virtual machine; detect crashes and restart automatically
Too many false alarms from the analyser	Model ownership hand-over and error codes; let the runtime stage refute the rest
kmemleak misses a leak	Repeat each test and clear stale memory (zeroing, stack erasing) so leaks are not hidden
The virtual machine is slow	Use KVM acceleration when available; only fail the calls the analyser points at
Real modules need hardware	Choose modules that run without hardware (virtual network devices, file systems, block simulators)

15. References

[1] Wang, W. “MLEE: Effective Detection of Memory Leaks on Early-Exit Paths in OS Kernels.” USENIX Annual Technical Conference (ATC), 2021, pp. 31–45.

[2] Linux kernel documentation: “Kernel Memory Leak Detector (kmemleak).” https://docs.kernel.org/dev-tools/kmemleak.html

[3] Linux kernel documentation: “Fault injection capabilities infrastructure.” https://docs.kernel.org/fault-injection/fault-injection.html

[4] Google. “syzkaller: an unsupervised coverage-guided kernel fuzzer.” https://github.com/google/syzkaller

[5] Emamdoost, N., Wu, Q., Lu, K., McCamant, S. “Detecting Kernel Memory Leaks in Specialized Modules with Ownership Reasoning (K-MELD).” NDSS, 2021.

[6] Lattner, C., Adve, V. “LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation.” CGO, 2004.

[7] Bellard, F. “QEMU, a Fast and Portable Dynamic Translator.” USENIX ATC, 2005.

[8] Cppcheck: a tool for static C/C++ code analysis. https://cppcheck.sourceforge.io/

[9] Silberschatz, A., Galvin, P. B., Gagne, G. Operating System Concepts, 10th ed. Wiley, 2018.

[10] Corbet, J., Rubini, A., Kroah-Hartman, G. Linux Device Drivers, 3rd ed. O’Reilly, 2005.